

TECNOLÓGICO DE ESTUDIOS SUPERIORES DE ECATEPEC

División de Ingeniería en Sistemas Computacionales

PROFESOR : JUAN ALBERTO FLORES GONZÁLEZ

EVOSYS – SISTEMA DE GESTIÓN EMPRESARIAL

EQUIPO 5 GRUPO : 5201

INTEGRANTES :

•BUENDÍA HERNÁNDEZ ÁNGEL YAHVE DAVID

•CHÁVEZ ARELLANO EDUARDO

•CRUZ ESTEVA ANDREA ALEJANDRA

•GARCÍA GARCÍA DAMIÁN ALBERTO

•MENA VELAZQUEZ JOLIET

•PELCASTRE CABALLERO IRVING GEOVANY

•VÁZQUEZ MARTÍNEZ JONATHAN GERARDO

Programación Orientada a Objetos

1.Resumen Ejecutivo

El presente proyecto describe el diseño, desarrollo e implementación del sistema "EVOSYS", una solución orientada a la gestión integral de clientes, productos y ventas. El objetivo principal es optimizar procesos administrativos mediante la automatización, reduciendo errores humanos y mejorando la eficiencia operativa. El sistema fue desarrollado utilizando Java como lenguaje principal y MySQL como gestor de base de datos. Se incorporaron prácticas como el uso de sentencias preparadas, validación de datos y manejo de transacciones, lo cual garantiza la integridad y seguridad de la información. Además, EVOSYS se distingue por ser un sistema accesible e intuitivo, diseñado para ser utilizado por cualquier persona sin conocimientos técnicos previos, lo que lo convierte en una propuesta innovadora dentro del mercado mexicano. Los resultados obtenidos demuestran que el sistema es funcional, eficiente, adaptable y con potencial de implementación en entornos reales.

2.INTRODUCCIÓN

La gestión eficiente de la información es un elemento fundamental en cualquier organización moderna. En muchos casos, los negocios pequeños carecen de herramientas tecnológicas que les permitan organizar sus datos de manera adecuada. Este proyecto surge como respuesta a dicha problemática, proponiendo un sistema capaz de integrar diferentes procesos en una sola plataforma. EVOSYS permite al usuario administrar información de manera clara, estructurada y accesible. El enfoque principal del proyecto es desarrollar una solución tecnológica accesible e innovadora dentro del mercado mexicano, permitiendo que cualquier usuario pueda utilizar el sistema sin requerir conocimientos técnicos avanzados.

Enfoque del Proyecto

EVOSYS no solo se centra en el desarrollo de un sistema funcional, sino en la creación de una herramienta tecnológica accesible, inclusiva e innovadora. Desarrollar un software que pueda ser utilizado por cualquier persona, sin importar su nivel de conocimiento en tecnología, representa una ventaja competitiva frente a otros sistemas tradicionales. Implementar una solución intuitiva permite reducir la brecha tecnológica existente en pequeños negocios, facilitando su digitalización y crecimiento.

3. Planteamiento del problema

Problemática general

En México, una gran cantidad de pequeños negocios operan sin sistemas digitales adecuados, lo que limita su crecimiento, organización y competitividad dentro del mercado. Problemática específica.

- Uso de métodos manuales.
- Errores en registros.
- Falta de control de inventarios.
- Sistemas complejos difíciles de usar.
- Baja accesibilidad tecnológica.
- Dependencia de conocimientos técnicos .

Enfoque del problema

Existe la necesidad de desarrollar un sistema accesible, intuitivo e innovador que permita a cualquier persona, sin conocimientos previos, administrar su negocio de forma eficiente. El proyecto busca marcar una diferencia mediante una solución fácil de usar, adaptable y enfocada al mercado mexicano.

4. Justificación

La implementación de EVOSYS permite mejorar significativamente la gestión administrativa mediante la automatización de procesos, la reducción de errores y la optimización de la organización de la información. El sistema está diseñado para ser accesible, funcional e innovador, lo que facilita su uso en pequeños negocios que no cuentan con herramientas tecnológicas avanzadas.

Impacto en el mercado

EVOSYS contribuye directamente al desarrollo del mercado mexicano al ofrecer una herramienta tecnológica accesible para pequeños negocios. Permite la digitalización de procesos tradicionales, mejorando la competitividad y eficiencia en el entorno comercial.

5.Objetivo General

Desarrollar e implementar un sistema de gestión empresarial en Java que automatice el control de ventas, inventario y clientes mediante Programación Orientada a Objetos, integrando interfaces gráficas y base de datos MySQL.

Objetivos Específicos

- Analizar las necesidades de los pequeños negocios.
- Diseñar la estructura del sistema utilizando Programación Orientada a Objetos.
- Desarrollar el sistema en Java.
- Implementar interfaces gráficas accesibles para todo tipo de usuario.
- Integrar una base de datos MySQL.
- Programar funcionalidades de registro y control.
- Validar el funcionamiento del sistema mediante pruebas.

5.1 Innovación del Proyecto

EVOSYS se diferencia de otros sistemas al enfocarse en la accesibilidad total para el usuario.

- Desarrollar un sistema utilizable sin conocimientos técnicos previos.
- Diseñar interfaces intuitivas.
- Adaptar el sistema a pequeños negocios mexicanos.
- Ofrecer actualizaciones constantes.
- Reducir la brecha tecnológica.

5.2 Costos y Beneficios

Análisis de Costos y Beneficios del Sistema

Costos	Beneficios
Desarrollo del sistema	Automatización de procesos
Implementación	Reducción de errores humanos
Mantenimiento	Mejor control de inventario
Actualizaciones anuales	Acceso rápido a la información
	Uso accesible para cualquier persona

6.Marco Teorico

Programación Orientada a Objetos (POO)

La Programación Orientada a Objetos es un paradigma de programación que permite estructurar el software en entidades llamadas clases, las cuales representan objetos del mundo real.

En el desarrollo de EVOSYS, se aplicó este paradigma para organizar el sistema en componentes independientes, facilitando su mantenimiento, reutilización y escalabilidad.

Los principales conceptos aplicados son:

- Encapsulamiento: proteger los datos mediante atributos privados y métodos públicos.
- Abstracción: representar elementos reales como clientes, productos y ventas.
- Herencia (conceptual): permitir la reutilización de código en estructuras similares.
- Polimorfismo (conceptual): posibilidad de utilizar métodos con diferentes comportamientos.

El uso de POO permitió desarrollar un sistema modular, ordenado y fácil de mantener.

Lenguaje de Programación Java

Java es un lenguaje de programación orientado a objetos ampliamente utilizado en el desarrollo de aplicaciones empresariales.

Se seleccionó Java debido a sus características:

- Portabilidad (multiplataforma).
- Robustez y seguridad.
- Gran comunidad y soporte.

Facilidad para trabajar con bases de datos.

En EVOSYS, Java permitió implementar la lógica del sistema de forma estructurada y confiable.

Bases de Datos Relacionales

Una base de datos relacional es un sistema que organiza la información en tablas relacionadas entre sí mediante claves primarias y foráneas.

En este proyecto se utilizó MySQL para almacenar:

- Clientes.
- Productos.
- Ventas.

Esto permitió mantener la información organizada, evitar duplicidades y realizar consultas eficientes.

MySQL como Sistema Gestor de Base de Datos

MySQL es un sistema gestor de bases de datos relacional ampliamente utilizado por su eficiencia, seguridad y facilidad de uso.

Se eligió MySQL porque:

- Permite manejar grandes volúmenes de datos.
- Es compatible con Java.
- Es confiable y rápido.
- Es ampliamente utilizado en sistemas empresariales.

Su implementación permitió almacenar y recuperar información de manera eficiente.

JDBC (Java Database Connectivity)

JDBC es una API que permite conectar aplicaciones Java con bases de datos.

En EVOSYS, JDBC se utilizó para:

- Establecer conexión con MySQL.
- Ejecutar consultas SQL.
- Insertar, actualizar y consultar datos.

Esto permitió integrar el sistema con la base de datos de forma eficiente.

Lenguaje SQL

SQL (Structured Query Language) es el lenguaje utilizado para gestionar bases de datos.

En este proyecto se utilizó para:

- Insertar registros.
- Consultar información.
- Actualizar datos.
- Eliminar registros.

El uso de SQL permitió manipular la información de manera estructurada.

Transacciones en Base de Datos

Las transacciones permiten ejecutar varias operaciones como una sola unidad de trabajo.

Esto asegura que:

- Si una operación falla, todas se cancelan.
- Se mantiene la integridad de los datos.

En EVOSYS, esto es importante en procesos como el registro de ventas.

Seguridad en Base de Datos

La seguridad es un aspecto fundamental en cualquier sistema.

En este proyecto se implementaron medidas como:

- Uso de PreparedStatement para evitar inyección SQL.
- Validación de datos de entrada.
- Control de operaciones en la base de datos.

Esto garantiza la protección de la información.

7. Metodología

El sistema EVOSYS fue desarrollado utilizando una metodología incremental, que posibilita la construcción del sistema por fases, con una evaluación constante de su rendimiento y mejoras continuas. Esta metodología se eligió porque posibilita identificar los errores con antelación, adecuar el sistema a nuevas exigencias y garantizar un desarrollo ordenado.

La metodología incremental implica dividir el desarrollo del sistema en partes diminutas y funcionales, conocidas como incrementos. Cada mejora pasa por las etapas de análisis, diseño, desarrollo e implementación, lo que hace posible obtener resultados intermedios que pueden ser evaluados antes de seguir con la próxima fase.

A lo largo de esta etapa, se implementó la metodología en el proyecto real, empezando por analizar las especificaciones. En primer lugar, se llevó a cabo un estudio exhaustivo de las necesidades de los pequeños negocios, con el objetivo de identificar las dificultades más importantes en la administración de clientes, inventarios y ventas. Se observó que numerosas empresas operan de manera manual, lo cual origina desorganización y equivocaciones. Los requerimientos funcionales del sistema se definieron a partir de esta investigación, y entre ellos se encuentran: la administración del inventario, el registro de los clientes, el registro de ventas y el registro de productos. La implementación práctica del sistema en el proyecto está compuesta por estas funciones.

Luego, se diseñó el sistema utilizando la técnica de Programación Orientada a Objetos (POO), estableciendo las clases esenciales como Cliente, Producto y Venta. Se establecieron los métodos y atributos de cada elemento, lo que asegura su adecuada organización. Además, se estableció una arquitectura de capas (de presentación, lógica y datos), lo que permite repartir las responsabilidades y facilita la escalabilidad y el mantenimiento del sistema. El diseño se llevó a cabo con la finalidad de desarrollar un sistema que fuera modular, escalable y fácil de comprender.



Desarrollo del Sistema en Java

Se implementó el sistema utilizando el lenguaje Java, debido a su robustez, portabilidad y amplia utilización en el desarrollo de software empresarial.

Se desarrollaron clases para representar cada entidad del sistema y se implementaron métodos para gestionar las operaciones principales.

Se aplicaron principios de Programación Orientada a Objetos como:

- Encapsulamiento.
- Abstracción.
- Modularidad.

Esto permitió construir un sistema organizado y reutilizable.

Implementación de Interfaces Gráficas

- Se desarrollaron interfaces gráficas con el objetivo de facilitar la interacción del usuario con el sistema.
- Las interfaces fueron diseñadas para ser intuitivas, permitiendo que cualquier persona pueda utilizar el sistema sin conocimientos técnicos.
- Se priorizó la simplicidad en el diseño para reducir la complejidad y mejorar la experiencia del usuario.

Integración con Base de Datos MySQL (Interfaces Grafica y explicacion de cada una de ellas)

Se implementó una base de datos en MySQL para almacenar la información del sistema.

Se crearon tablas para clientes, productos y ventas.

Se utilizó JDBC para establecer la conexión entre Java y la base de datos.

Se aplicaron sentencias preparadas (PreparedStatement) para evitar vulnerabilidades como la inyección SQL.

Esto permitió garantizar la seguridad e integridad de los datos.

Pruebas del Sistema

Se realizaron pruebas funcionales para verificar que cada módulo del sistema operara correctamente.

Se probaron procesos como:

- Registro de clientes.
- Registro de productos.
- Realización de ventas.
- Actualización de inventario.

Estas pruebas permitieron detectar errores y corregirlos antes de la entrega final.

8. Arquitectura del sistema

El sistema EVOSYS se diseñó bajo una arquitectura de tres capas, con el objetivo de organizar el código, facilitar su mantenimiento y permitir futuras mejoras sin afectar el funcionamiento general del sistema.

Las capas que conforman la arquitectura son:

Capa de Presentación

Encargada de la interacción con el usuario mediante interfaces gráficas. Permite al usuario ingresar datos, visualizar información y ejecutar acciones dentro del sistema.

Se diseñó con un enfoque intuitivo, priorizando la facilidad de uso para personas sin conocimientos técnicos.

Capa de Lógica de Negocio

Esta capa contiene las reglas del sistema y el procesamiento de datos.

Aquí se implementan las operaciones principales como:

- Registro de clientes.
- Control de inventario.
- Procesamiento de ventas.

Permite que el sistema funcione de manera estructurada y organizada.

Capa de Datos

Encargada del almacenamiento de la información en la base de datos MySQL.

Se gestionan operaciones como:

- Inserción de datos.
- Consultas.
- Actualizaciones.

Esto garantiza la persistencia y seguridad de la información.

Importancia de la arquitectura

El uso de esta arquitectura permite:

- Separar responsabilidades.
- Facilitar mantenimiento.
- Mejorar escalabilidad.
- Evitar errores estructural.

Escalabilidad del Sistema

El sistema EVOSYS fue diseñado para permitir su crecimiento sin necesidad de modificar completamente su estructura.

Gracias al modularidad del sistema, es posible:

- Agregar nuevos módulos.
- Integrar nuevas funcionalidades.
- Adaptar el sistema a diferentes tipos de negocios.

Esto permite que el sistema evolucione conforme a las necesidades del usuario.

9.Desarrollo del Proyecto

El sistema EVOSYS está estructurado mediante clases en Java, siguiendo el paradigma de Programación Orientada a Objetos. A continuación, se describe de manera detallada cada una de las clases implementadas y su función dentro del sistema.

Clase Cliente

La Clase Cliente representa a cada persona que compra o interactúa con el sistema. Es decir, aquí se guarda toda la información individual de un cliente.

Atributos:

- idCliente: Es un identificador único para cada cliente (como una clave).
- nombre: Guarda el nombre completo del cliente.
- teléfono: Permite tener un medio de contacto.
- dirección: Indica dónde vive el cliente.
- RFC: Es un dato fiscal que sirve para facturación.

Función:

Su función principal es almacenar y organizar la información de cada cliente, permitiendo que el sistema tenga datos claros y ordenados para futuras consultas o procesos como ventas y facturación.

Clase Clientes

La Clase Clientes se encarga de manejar a todos los clientes en conjunto. No representa a uno solo, sino que administra toda la lista.

Métodos:

- agregarCliente(): Permite registrar un nuevo cliente en el sistema.
- modificarCliente(): Sirve para actualizar datos de un cliente existente.
- buscarCliente(): Permite encontrar un cliente usando algún dato (como ID o nombre).
- mostrarClientes(): Muestra todos los clientes registrados.

Importancia:

Es importante porque centraliza toda la administración de los clientes, facilitando su control, organización y acceso a la información.

Clase ConexionBD

La Clase ConexionBD es la encargada de conectar el programa con la base de datos (MySQL).

Funciones:

- Establecer conexión: Permite que Java se conecte con la base de datos.
- Gestionar acceso a la base de datos: Controla las consultas, inserciones, actualizaciones y eliminaciones de datos.

Importancia:

Sin esta clase, el sistema no podría guardar ni recuperar información. Es esencial porque permite la comunicación entre el programa y la base de datos.

Clase Producto

La Clase Producto representa cada artículo que se vende o se tiene en inventario.

Atributos:

- idProducto: Identificador único del producto.
- nombre: Nombre del producto.
- stock: Cantidad disponible en inventario.
- precio: Costo del producto.

Función:

Permite almacenar la información básica de cada producto, facilitando su control dentro del sistema.

Clase Productos

La Clase Productos administra todos los productos del sistema, es decir, el inventario completo.

Funciones:

- Registrar productos: Permite agregar nuevos productos al sistema.
- Actualizar stock: Modifica la cantidad disponible cuando hay ventas o entradas de mercancía.
- Consultar inventario: Permite ver todos los productos y su disponibilidad.

Importancia:

Es fundamental porque permite llevar el control del inventario, evitando errores como falta de productos o sobreventa.

Clase Venta

La Clase Venta representa una transacción que ocurre cuando un cliente compra un producto. Es decir, guarda toda la información de una compra específica.

Atributos:

- cliente: Indica qué cliente realizó la compra.
- producto: Es el producto que se vendió.
- cantidad: Número de unidades compradas.
- total: Monto total de la venta (precio × cantidad).
- fecha: Día en que se realizó la transacción.

Función:

Su función es registrar y almacenar cada venta realizada, permitiendo llevar un control de las operaciones comerciales del sistema.

Clase VentasClientes

La Clase VentasClientes se encarga de gestionar todo el proceso de venta, es decir, controla desde que se realiza la compra hasta que se actualiza el inventario.

Funciones:

- Registrar venta:
- Guarda la información de la venta en el sistema utilizando los datos del cliente y producto.
- Validar stock:
- Verifica que haya suficiente cantidad del producto antes de realizar la venta. Esto evita vender productos que no están disponibles.
- Actualizar inventario:
- Después de la venta, descuenta la cantidad vendida del stock disponible.

Importancia:

Es muy importante porque asegura que las ventas se realicen correctamente, evitando errores como vender productos sin existencia y manteniendo actualizado el inventario.

Clase Sistema Tienda Main

La Clase Sistema Tienda Main es la clase principal del sistema. Es donde inicia y se ejecuta todo el programa.

Funciones:

- Mostrar menú:
- Presenta las opciones disponibles al usuario (por ejemplo: agregar cliente, registrar venta, ver productos, etc.).
- Coordinar módulos:
- Se encarga de conectar todas las clases (Clientes, Productos, Ventas, etc.) para que trabajen juntas.
- Ejecutar el sistema:
- Inicia el programa y mantiene su funcionamiento hasta que el usuario decide salir.

Importancia:

Es fundamental porque controla todo el sistema, actuando como el centro desde donde se ejecutan todas las operaciones.

10.Explicación Técnica del Código

El sistema está desarrollado en Java utilizando clases, métodos y estructuras de control que permiten ejecutar las operaciones del sistema.

Se utiliza JDBC para conectar con MySQL, permitiendo ejecutar consultas SQL para manipular la información.

El código está estructurado de forma modular, facilitando su mantenimiento.

Análisis Técnico del Sistema

El uso de Java permite desarrollar un sistema robusto, portable y seguro.

MySQL garantiza el almacenamiento eficiente de datos.

La combinación de POO y base de datos permite un sistema organizado, escalable y funcional.

11.Flujo del Sistema (FALTA DIAGRAMA DE FLUJO)

1. El usuario inicia el sistema.
2. Selecciona una opción del menú.
3. Ingresa datos.
4. El sistema valida la información.
5. Se ejecuta la operación.
6. Se muestra el resultado.
7. Completa.

12. Casos de uso (falta los diagramas de caso de uso, se sugiere utilizar algún software para el uso de casos)

Registrar Cliente.

- Permite almacenar información de nuevos clientes.

Registrar Venta.

- Permite realizar una transacción completa.

13. Ejemplo de ejecución

Cliente: Juan.

Producto: Laptop.

Cantidad: 1.

El sistema valida stock, registra la venta y actualiza el inventario.

14. Resultados

Mejora en la organización

La implementación del sistema permite una organización mucho más eficiente de la información, tanto de clientes como de productos y ventas.

Antes, los datos podían estar desordenados o incluso perderse, pero ahora todo se encuentra estructurado en clases y almacenado en una base de datos.

Esto facilita:

- Acceder rápidamente a la información.
- Mantener un control ordenado de cada cliente y producto.
- Evitar duplicidad de datos.

En general, el sistema ayuda a que la gestión del negocio sea más clara, ordenada y profesional.

Reducción de errores

Uno de los principales resultados es la disminución de errores humanos, ya que el sistema automatiza y valida muchos procesos.

Por ejemplo:

- Al validar el stock, se evita vender productos que no están disponibles.
- Los cálculos del total se hacen automáticamente, evitando errores matemáticos.
- La información se guarda correctamente sin omisiones o confusiones.

Esto genera mayor confiabilidad en el sistema y evita problemas tanto para el negocio como para los clientes.

Automatización de procesos

El sistema permite que muchas tareas que antes se hacían manualmente ahora se realicen de forma automática.

Algunos ejemplos:

- Registro automático de ventas.
- Actualización inmediata del inventario.
- Búsqueda rápida de clientes y productos.

Gracias a esto:

- Se ahorra tiempo.
- Se reduce el esfuerzo del usuario.
- Se mejora la eficiencia del negocio.

La automatización hace que el sistema sea más rápido, práctico y funcional en el día a día.

Evaluación del Sistema

El sistema cumple con los objetivos planteados, mostrando eficiencia y facilidad de uso.

15.Análisis

El sistema demuestra ser una solución viable para mejorar la gestión empresarial.

Análisis de Impacto

EVOSYS permite la digitalización de pequeños negocios, facilitando su crecimiento.

16.Conclusiones

El sistema cumple con los objetivos, destacando por su accesibilidad y funcionalidad.

17.Recomendaciones

- Mejorar interfaz: Optimizar la navegación y el diseño visual aplicando principios de usabilidad, además de realizar pruebas con usuarios para reducir errores y mejorar la experiencia.
- Implementar seguridad avanzada: Incorporar cifrado de datos, autenticación segura y control de accesos, así como respaldos periódicos para proteger la información.

18.Limitaciones

- Dependencia local.
- Interfaz básica.

Riesgos del Sistema

- Fallas técnicas.
- Dependencia de base de dato.

19.Trabajo futuro

- Migración a web.
- App móvil.

Mejora Continua

- Optimización del sistema.
- Nuevas funcionalidades.

20. Glosario

- SQL: lenguaje de base de datos.
- JDBC: conexión Java-BD.
- Stock: inventario.

21. Referencias

- Oracle. (2023). Java Documentation.
- MySQL. (2023). MySQL Documentation.

22. Anexos

Seguridad del Sistema.

Se implementaron medidas como:

- PreparedStatement
- Validación de datos.
- Control de acceso.

DOCUMENTACIÓN DEL SISTEMA – CLASE PRINCIPAL

La clase SistemaTiendaMain representa el punto de entrada principal del sistema de tienda, siendo la encargada de iniciar la ejecución del programa, establecer la conexión con la base de datos y coordinar la interacción del usuario con los diferentes módulos del sistema.

Esta clase actúa como el controlador central, ya que dirige el flujo del programa hacia los módulos de productos, clientes y ventas.

IDENTIDAD DEL SISTEMA

```
private static final String EMPRESA = "EVOSYS";  
private static final String VERSION = "v1.3 - SQL Enterprise";
```

Estas constantes representan el nombre de la empresa y la versión del sistema, utilizadas para mostrar información al usuario.

MÉTODO MAIN

```
public static void main(String[] args)
```

Es el punto de inicio del programa.

CONEXIÓN A LA BASE DE DATOS

```
ConexionBD objetoConexion = new ConexionBD();  
objetoConexion.establecerConexion();
```

Permite conectar el sistema con MySQL.

CREACIÓN DE OBJETOS

```
Productos productos = new Productos();  
Clientes clientes = new Clientes();  
VentasClientes ventas = new VentasClientes();
```

Se instancian los módulos principales.

Mensaje de bienvenida

```
JOptionPane.showMessageDialog(...)
```

Muestra un mensaje inicial al usuario.

MENÚ PRINCIPAL

```
do { ... } while (!"4".equals(opcionPrincipal));
```

Permite que el sistema siga ejecutándose hasta que el usuario decida salir.

SWITCH PRINCIPAL

Permite navegar entre módulos:

- Productos
- Clientes
- Ventas

MÉTODO MENUPRODUCTOS

Permite:

- Agregar productos
- Modificar
- Mostrar inventario
- Importar CSV

MÉTODO MENUCLIENTES

Permite:

- Agregar clientes
- Modificar
- Mostrar lista
- Importar CSV

MÉTODO MENUVENTAS PERMITE:

- Registrar ventas
- Ver historial
- Ver resumen

FLUJO GENERAL

1. Se inicia el sistema
2. Se conecta a la base de datos
3. Se muestran menús
4. El usuario interactúa
5. Se ejecutan operaciones

CLASE CLIENTE

ATRIBUTOS

```
public int idCliente;  
public String nombre;  
public String telefono;  
public String direccion;  
public String rfc;
```

CONSTRUCTORES

```
public Cliente() {}  
  
public Cliente(int idCliente, String nombre, String telefono)
```

Permite almacenar información del cliente para su uso en ventas.

La clase Cliente tiene como propósito representar a un cliente dentro del sistema. Es una clase de tipo modelo, ya que únicamente almacena información relacionada con cada cliente, como su identificador, nombre, teléfono, dirección y RFC.

Esta clase no realiza operaciones por sí misma, sino que sirve como estructura de datos que puede ser utilizada por otras clases para manipular la información de los clientes. Es fundamental para poder registrar ventas, ya que permite identificar quién realiza cada compra

La clase Cliente representa a una persona o entidad que interactúa con el sistema de tienda realizando compras. Es una clase de tipo modelo, lo que significa que su función principal es almacenar datos y servir como estructura para representar a los clientes dentro del sistema.

Esta clase permite organizar la información de cada cliente de forma clara, facilitando su uso en otras partes del sistema, como el módulo de ventas.

Importancia dentro del sistema

La clase Cliente es importante porque:

- Permite identificar a cada cliente de manera única
- Almacena la información necesaria para realizar ventas
- Sirve como base para registrar transacciones
- Facilita la gestión de datos en la base de datos

Sin esta clase, no sería posible asociar una venta a una persona específica, lo que limitaría el control del sistema.

Estructura de datos

La clase está compuesta por varios atributos que representan la información del cliente.

ID Identificador del cliente

El atributo idCliente es un número entero que permite identificar de manera única a cada cliente dentro del sistema.

Este identificador es importante porque:

- Evita duplicidad de registros
- Permite realizar búsquedas rápidas
- Funciona como clave primaria en la base de datos

Nombre del cliente

El atributo nombre almacena el nombre del cliente.

Su importancia radica en que:

- Permite identificar al cliente de manera visual
- Es utilizado en consultas y búsquedas
- Se muestra en ventas y reportes

Teléfono

El atributo telefono guarda el número telefónico del cliente.

Este dato es útil para:

- Contactar al cliente
- Mantener un registro de comunicación
- Identificar clientes en el sistema

Dirección

El atributo direccion permite almacenar la ubicación del cliente.

Esto puede ser útil para:

- Envíos o entregas
- Registro completo del cliente
- Sistemas más avanzados de logística

RFC

El atributo rfc representa el Registro Federal de Contribuyentes.

Su uso es importante en:

- Facturación
- Control fiscal
- Sistemas empresariales más completos

CLASE CLIENTES

La clase Clientes se encarga de gestionar todas las operaciones relacionadas con los clientes dentro del sistema. Su función principal es interactuar con la base de datos para realizar acciones como agregar nuevos clientes, modificar su información, buscarlos y mostrarlos en pantalla.

Además, permite importar clientes desde archivos CSV, lo que facilita la carga masiva de datos. Esta clase actúa como intermediaria entre el usuario y la base de datos, asegurando que la información de los clientes se almacene y se consulte correctamente.

agregarCliente()

Inserta un cliente en la base de datos usando:

```
INSERT INTO clientes (nombre, telefono) VALUES (?, ?);
```

modificarCliente()

```
UPDATE clientes SET nombre = ?, telefono = ? WHERE id_cliente = ?;
```

buscarCliente()

Busca por nombre:

```
SELECT * FROM clientes WHERE nombre LIKE ?;
```

mostrarClientes()

Muestra todos los clientes.

```
SELECT * FROM clientes;
```

importarClientesCSV()

- Lee archivo CSV
- Inserta datos en lote
- Usa executeBatch()

Actúa como un intermediario entre la interfaz del usuario y la base de datos, asegurando que las acciones realizadas por el usuario se reflejen correctamente en el sistema.

Importancia dentro del sistema

La clase Clientes tiene un papel fundamental en el sistema, ya que permite:

- Registrar nuevos clientes
- Actualizar la información existente
- Consultar datos de clientes
- Mostrar listados completos
- Integrar clientes en el proceso de ventas

Sin esta clase, el sistema no podría identificar a los usuarios que realizan compras, lo cual afectaría directamente el funcionamiento del módulo de ventas.

Funcionalidades principales

La clase incluye múltiples métodos que permiten gestionar los clientes de manera eficiente.

Registro de clientes

Permite agregar nuevos clientes mediante formularios interactivos. El usuario introduce los datos y estos se almacenan en la base de datos.

Esta funcionalidad es esencial para:

- Crear registros nuevos
- Mantener actualizada la base de datos
- Iniciar el proceso de ventas

Modificación de clientes

Permite actualizar los datos de un cliente existente, como su nombre o teléfono.

Esto es importante porque:

- Corrige errores
- Mantiene información actualizada
- Mejora la calidad de los datos

Búsqueda de clientes

Permite localizar clientes mediante su nombre.

Esta función es útil para:

- Verificar si un cliente existe
- Obtener información rápidamente
- Evitar duplicados

CLASE CONEXION BD

La clase ConexionBD tiene la función de establecer la conexión entre el sistema y la base de datos MySQL. Es una de las clases más importantes, ya que permite que el programa pueda guardar, consultar y modificar información en la base de datos.

Centraliza toda la configuración necesaria para conectarse, como la dirección del servidor, el nombre de la base de datos, el usuario y la contraseña. Gracias a esta clase, las demás clases no necesitan preocuparse por cómo conectarse, sino que simplemente utilizan su método para acceder a la base de datos.

CONFIGURACIÓN

```
private final String bd = "evosys";  
private final String url = "jdbc:mysql://localhost:3306/" + bd;  
private final String usuario = "root";  
private final String password = "*****";
```

MÉTODO CONECTAR

```
public Connection establecerConexion()
```

- Carga driver
- Establece conexión
- Maneja errores

El método `establecerConexion` es fundamental para el correcto funcionamiento del sistema y se encarga de los siguientes pasos:

Carga del driver: Utiliza la clase `Class.forName()` para cargar el driver de MySQL necesario para establecer la conexión. Esto permite que la aplicación pueda comunicarse correctamente con la base de datos.

Establecimiento de la conexión: Mediante la clase `DriverManager` y el método `getConnection()`, se establece la conexión utilizando la URL, el usuario y la contraseña previamente configurados. Si la conexión se establece correctamente, se devuelve un objeto `Connection` que puede ser utilizado por las demás partes del programa.

Manejo de errores: Se utilizan bloques `try-catch` para manejar posibles excepciones que puedan ocurrir durante el proceso de conexión. Esto incluye la posibilidad de un `ClassNotFoundException` al cargar el driver o un `SQLException` al intentar conectarse a la base de datos.

Al proporcionar una interfaz simplificada para la conexión, la clase `ConexionBD` facilita el desarrollo y mantenimiento del sistema, asegurando que los desarrolladores puedan centrarse en las funcionalidades específicas de su aplicación sin preocuparse por los detalles de la conexión a la base de datos.

Importancia dentro del sistema

Esta clase es esencial porque:

- Permite la comunicación con la base de datos
- Hace posible guardar y consultar información
- Centraliza la configuración de conexión
- Evita duplicación de código en otras clases

Funcionamiento de la conexión

El proceso de conexión incluye:

1. Cargar el driver de MySQL
2. Establecer conexión mediante DriverManager
3. Manejar errores en caso de fallo

Uso en el sistema

Es utilizada por:

- Clientes
- Productos
- VentasClientes

Todas las clases que requieren acceso a la base de datos dependen de esta.

CLASE PRODUCTO

La clase Producto representa un producto dentro del sistema de tienda. Su función es almacenar la información de cada artículo disponible para la venta, como su identificador, nombre, cantidad en stock y precio.

Al igual que la clase Cliente, esta es una clase de tipo modelo, lo que significa que se utiliza como estructura de datos. Es fundamental para el control del inventario, ya que permite saber qué productos existen, cuántos hay disponibles y cuánto cuestan.

COMPONENTES DE LA CLASE

Atributos

Datos del producto.

```
public int idProducto;  
public String nombre;  
public int stock;  
public double precio;
```

CONSTRUCTOR

Inicializa valores

```
public Producto(int idProducto, String nombre, int stock, double precio)
```

FRAGMENTO IMPORTANTE

```
this.precio = precio;
```

Asigna el precio.

La clase Producto es una de las estructuras fundamentales del sistema, ya que representa cada uno de los artículos que pueden ser almacenados, gestionados y vendidos dentro de la tienda. Su correcta implementación permite llevar un control preciso del inventario, lo cual es esencial para el funcionamiento adecuado del sistema.

Cada objeto de esta clase contiene información relevante sobre un producto específico, permitiendo identificarlo de manera única y conocer sus características principales, como su nombre, precio y cantidad disponible en stock.

La clase Productos es responsable de gestionar el inventario del sistema. A través de esta clase se pueden realizar operaciones como agregar, modificar, consultar y actualizar productos.

Importancia

- Controla el inventario
- Permite registrar productos
- Evita errores en ventas por falta de stock

Manejo del inventario

El sistema controla:

- Cantidad disponible
- Actualización automática
- Validación antes de vender

Relación con otras clases

- Producto
- VentasClientes
- ConexionBD

CLASE PRODUCTOS

La clase Producto representa un producto dentro del sistema de tienda. Su función es almacenar la información de cada artículo disponible para la venta, como su identificador, nombre, cantidad en stock y precio.

Al igual que la clase Cliente, esta es una clase de tipo modelo, lo que significa que se utiliza como estructura de datos. Es fundamental para el control del inventario, ya que permite saber qué productos existen, cuántos hay disponibles y cuánto cuestan.

Método agregarProducto()

Inserta productos.

Método modificarProducto()

Actualiza datos.

Método mostrarProductos()

Muestra inventario.

Fragmento de código importante

INSERT INTO productos (nombre, precio, stock)

VALUES (?, ?, ?);

Este fragmento de código SQL es esencial para agregar nuevos productos a la base de datos. Utiliza parámetros para insertar el nombre del producto, su precio y la cantidad disponible en el inventario, asegurando que la información se guarde correctamente. Es importante validar los datos antes de la inserción para evitar errores y mantener la integridad del sistema. Además, se recomienda implementar transacciones para garantizar que las operaciones sean seguras y confiables, especialmente en un entorno de ventas donde la precisión del inventario es crítica.

Su función principal es actuar como intermediaria entre el usuario y la base de datos, permitiendo realizar operaciones como agregar nuevos productos, modificar los existentes, consultar el inventario y mantener actualizado el stock.

Importancia dentro del sistema

La clase Productos es fundamental para el correcto funcionamiento del sistema, ya que:

- Permite registrar los productos disponibles para la venta
- Mantiene actualizado el inventario
- Evita errores al momento de vender productos sin stock
- Facilita la administración de los artículos

Sin esta clase, el sistema no tendría control sobre los productos, lo que afectaría directamente las ventas y la organización del negocio.

Funcionalidades principales

Esta clase implementa diversas funciones que permiten gestionar el inventario de manera eficiente.

Registro de productos

Permite agregar nuevos productos al sistema mediante la captura de datos como nombre, precio y cantidad en stock.

Estos datos son almacenados en la base de datos mediante consultas SQL, lo que asegura su persistencia.

Modificación de productos

Permite actualizar la información de un producto existente, como su nombre, precio o stock.

Esta funcionalidad es importante para mantener la información actualizada y corregir posibles errores.

Visualización del inventario

Permite mostrar todos los productos registrados en la base de datos.

Esto facilita:

- Revisar el inventario disponible
- Identificar productos existentes
- Consultar precios y cantidades

Control de stock

Uno de los aspectos más importantes de esta clase es el control del stock.

Permite:

- Verificar si hay suficientes unidades antes de una venta
- Reducir automáticamente el stock cuando se realiza una venta
- Evitar ventas con inventario insuficiente

CLASE: VENTA

La clase Venta representa una transacción realizada dentro del sistema. Su función es almacenar toda la información relacionada con una venta, incluyendo el cliente que realizó la compra, el producto vendido, la cantidad de unidades, el total de la venta y la fecha.

Esta clase permite estructurar la información de las ventas para que pueda ser almacenada en la base de datos y posteriormente consultada en reportes o historiales.

COMPONENTES

Atributos

```
public int idVenta;  
public int idCliente;  
public int idProducto;  
public int unidades;  
public double totalVenta;
```

FRAGMENTO IMPORTANTE

```
totalVenta = unidades * precio;
```

Cálculo del total.

Además, la clase Venta incluye métodos que facilitan la gestión y el acceso a los datos de cada transacción. Estos métodos podrían incluir:

Métodos

- `public void registrarVenta():` Este método se encarga de registrar una nueva venta en el sistema, asegurándose de que todos los datos necesarios estén correctamente almacenados.
- `public double calcularImpuesto(double tasaImpuesto):` Calcula el impuesto aplicado a la venta según una tasa específica, permitiendo obtener un valor más preciso del total.
- `public String generarRecibo():` Genera un recibo detallado para el cliente, incluyendo la información de la venta, el cliente, y el desglose del precio.
- `public void actualizarInventario():` Actualiza el inventario del producto vendido, restando las unidades vendidas para mantener un control adecuado de stock.

Con estos componentes y métodos, la clase Venta se convierte en una herramienta esencial para el manejo eficiente de las transacciones comerciales, garantizando que toda la información relevante se registre de manera precisa y accesible.

La clase Venta representa una operación comercial dentro del sistema. Permite registrar información sobre una transacción realizada entre un cliente y un producto.

Importancia

- Permite registrar ingresos
- Almacena historial de ventas
- Facilita análisis del negocio

Datos que maneja

- Cliente
- Producto
- Cantidad
- Total
- Fecha

Uso en el sistema

Es utilizada por la clase VentasClientes para almacenar información de las transacciones.

CLASE VENTAS CLIENTES

La clase VentasClientes es la encargada de gestionar todo el proceso de ventas dentro del sistema. Es una de las clases más importantes, ya que integra la información de clientes y productos para registrar una venta completa.

Se encarga de validar que el cliente exista, verificar que el producto tenga suficiente stock, calcular el total de la venta, registrar la transacción en la base de datos y actualizar el inventario. También permite consultar el historial de ventas y generar resúmenes, lo que la convierte en el núcleo funcional del sistema.

COMPONENTES

Método agregarVenta()

Registra una venta.

Método mostrarVentas()

Muestra historial.

Método resumenVentasPorFecha()

Genera reportes.

Fragmento importante

SELECT * FROM ventas

Consulta para obtener ventas.

UPDATE productos SET stock = stock - ?

Actualiza inventario.

La clase Clientes es la encargada de gestionar toda la información relacionada con los clientes dentro del sistema de tienda. A diferencia de la clase Cliente, que únicamente representa los datos, esta clase contiene la lógica necesaria para interactuar con la base de datos y realizar operaciones sobre dichos datos.

Esta clase funciona como un intermediario entre el usuario y la base de datos, permitiendo que las acciones realizadas desde la interfaz gráfica se reflejen correctamente en el sistema.

La clase Clientes es fundamental porque:

- Permite registrar nuevos clientes en el sistema
- Facilita la actualización de datos existentes
- Permite consultar información de clientes
- Hace posible la integración con el módulo de ventas

Sin esta clase, el sistema no podría gestionar correctamente la información de los clientes, lo que afectaría directamente la capacidad de realizar ventas.

Funcionalidades principales

Esta clase incluye varias funciones clave que permiten administrar los clientes:

Registro de clientes

Permite agregar nuevos clientes mediante formularios interactivos. Los datos ingresados por el usuario son enviados a la base de datos mediante consultas SQL.

Modificación de datos

Permite actualizar la información de un cliente existente, como su nombre o teléfono. Esto es importante para mantener los datos actualizados.

Búsqueda de clientes

Facilita localizar clientes mediante su nombre. Esta función es útil cuando se necesita verificar si un cliente ya está registrado en el sistema.

Visualización

Permite mostrar una lista completa de todos los clientes registrados, lo cual es útil para consultas generales.

Importación desde CSV

Una de las funciones más importantes es la capacidad de importar clientes desde archivos CSV, lo que permite cargar grandes cantidades de datos de manera rápida y eficiente.

Interacción con la base de datos

La clase Clientes utiliza sentencias SQL para interactuar con la base de datos.

Algunas operaciones incluyen:

- Inserción de datos (INSERT)
- Actualización (UPDATE)
- Consulta (SELECT)

Estas operaciones se realizan mediante el uso de PreparedStatement, lo cual mejora la seguridad y eficiencia del sistema.

CONCLUSION

El desarrollo del sistema de gestión de tienda presentado en este trabajo permite comprender de manera integral la importancia de la programación orientada a objetos aplicada a un entorno real. A través de la implementación de distintas clases como clientes, productos y ventas, se logra estructurar un sistema funcional capaz de administrar información de manera eficiente y organizada.

Uno de los aspectos más relevantes de este proyecto es la correcta separación de responsabilidades entre las clases. Cada clase cumple una función específica dentro del sistema, lo cual facilita su comprensión, mantenimiento y escalabilidad. Por ejemplo, las clases modelo como Cliente, Producto y Venta se encargan de representar los datos, mientras que las clases como Clientes, Productos y VentasClientes gestionan la lógica del sistema y la interacción con la base de datos.

La conexión con la base de datos mediante la clase ConexionBD representa un elemento clave, ya que permite almacenar la información de manera persistente. Esto significa que los datos no se pierden al cerrar el programa, lo cual es fundamental en cualquier sistema real. Además, el uso de consultas SQL y herramientas como PreparedStatement demuestra una correcta implementación de buenas prácticas en el manejo de datos, especialmente en temas de seguridad.

Otro punto importante es la integración de todos los módulos del sistema. El programa no funciona de manera aislada, sino que cada componente se conecta con los demás para lograr un flujo completo. Por ejemplo, el módulo de ventas depende directamente de los clientes y productos, lo que refleja una estructura bien organizada y coherente.

Asimismo, el sistema incluye funcionalidades que simulan un entorno real de negocio, como el control de inventario, la validación de stock antes de realizar una venta y la generación de reportes. Estas características no solo demuestran el funcionamiento técnico del sistema, sino que también reflejan su utilidad práctica en situaciones reales.

A nivel técnico, este proyecto permite aplicar conceptos fundamentales como el uso de clases, objetos, métodos, estructuras de control, manejo de excepciones y conexión a bases de datos. Todo esto contribuye a fortalecer las habilidades de programación y a entender cómo se construyen sistemas más complejos.

ANEXO

Código fuente del sistema EVOSYS.

1. Main

```
SistemaTiendaMain.java X
Source History
1 package sistematienda;
2
3 import javax.swing.*;
4
5 public class SistemaTiendaMain {
6
7     // Identidad de la Empresa EVOSYS
8     private static final String EMPRESA = "EVOSYS";
9     private static final String VERSION = "v1.3 - SQL Enterprise";
10
11     public static void main(String[] args) {
12         // Inicializa la conexión a MySQL
13         ConexionBD objetoConexion = new ConexionBD();
14         objetoConexion.establecerConexion();
15
16         Productos productos = new Productos();
17         Clientes clientes = new Clientes();
18         VentasClientes ventas = new VentasClientes();
19
20         SistemaTiendaMain app = new SistemaTiendaMain();
21
22         // Bienvenida Corporativa
23         JOptionPane.showMessageDialog(null,
24             "Bienvenido a los Sistemas de Gestión " + EMPRESA + "\nIniciando módulos de base de datos...",
25             EMPRESA + " " + VERSION,
26             JOptionPane.INFORMATION_MESSAGE);
27
28         String opcionPrincipal;
29         do {
30             opcionPrincipal = JOptionPane.showInputDialog(
31                 null,
32                 "---- " + EMPRESA + " | PÁNEL PRINCIPAL ----\n\n" +
33                 "1. Gestión de Productos\n" +
34                 "2. Gestión de Clientes\n" +
35                 "3. Gestión de Ventas\n" +
36                 "4. Salir del Sistema\n" +
37                 "Seleccione una opción",
38                 EMPRESA,
39                 JOptionPane.QUESTION_MESSAGE
40             );
41
42             if (opcionPrincipal == null) break;
43
44             switch (opcionPrincipal) {
45                 case "1":
46                     app.menuProductos(productos);
47                     break;
```

```
SistemaTiendaMain.java X
Source History
47         break;
48         case "2":
49             app.menuClientes(clientes);
50             break;
51         case "3":
52             app.menuVentas(ventas, productos);
53             break;
54         case "4":
55             JOptionPane.showMessageDialog(null, "Cierre de sesión exitoso.\nEVOSYS: Tecnología que evoluciona contigo.", EMPRESA, 1);
56             break;
57         default:
58             JOptionPane.showMessageDialog(null, "Opción no válida", EMPRESA, 0);
59     } while (!"4".equals(opcionPrincipal));
60 }
61
62 public void menuProductos(Productos productos) {
63     String opcion;
64     do {
65         opcion = JOptionPane.showInputDialog(
66             null,
67             "---- " + EMPRESA + " | MÓDULO DE PRODUCTOS ----\n\n" +
68             "1. Agregar producto manual\n" +
69             "2. Modificar producto o stock\n" +
70             "3. Mostrar inventario completo\n" +
71             "4. IMPORTAR DESDE EXCEL (CSV)\n" +
72             "5. Volver al menú principal\n" +
73             "Powered by EVOSYS",
74             "Gestión de Productos",
75             JOptionPane.PLAIN_MESSAGE
76         );
77     } while (!"5".equals(opcion));
78
79     if (opcion == null) break;
80
81     switch (opcion) {
82         case "1": productos.agregarProducto(); break;
83         case "2": productos.modificarProducto(); break;
84         case "3": productos.mostrarProductos(); break;
85         case "4": productos.cargarDesdeExcelCSV(); break;
86         case "5": break;
87         default: JOptionPane.showMessageDialog(null, "Elija un número del 1 al 5.");
88     }
89 }
90
91 public void menuClientes(Clientes clientes) {
92     String opcion;
```

```
SistemaTiendaMain.java X
Source History
92     public void menuClientes(Clientes clientes) {
93         String opcion;
94         do {
95             opcion = JOptionPane.showInputDialog(
96                 null,
97                 "---- " + EMPRESA + " | MÓDULO DE CLIENTES ----\n\n" +
98                 "1. Añadir nuevo cliente manual\n" +
99                 "2. Actualizar datos de cliente\n" +
100                 "3. Ver lista de clientes\n" +
101                 "4. IMPORTAR LISTA DE CLASES (CSV)\n" +
102                 "5. Volver al menú principal\n" +
103                 "Powered by EVOSYS",
104                 "Gestión de Clientes",
105                 JOptionPane.PLAIN_MESSAGE
106             );
107
108             if (opcion == null) break;
109
110             switch (opcion) {
111                 case "1": clientes.agregarCliente(); break;
112                 case "2": clientes.modificarCliente(); break;
113                 case "3": clientes.mostrarClientes(); break;
114                 case "4": clientes.importarClientesCSV(); break;
115                 case "5": break;
116                 default: JOptionPane.showMessageDialog(null, "Elija un número del 1 al 5.");
117             }
118         } while (!"5".equals(opcion));
119     }
120
121     public void menuVentas(VentasClientes ventas, Productos productos) {
122         String opcion;
123         do {
124             opcion = JOptionPane.showInputDialog(
125                 null,
126                 "---- " + EMPRESA + " | MÓDULO DE VENTAS ----\n\n" +
127                 "1. Registrar nueva venta\n" +
128                 "2. Ver historial de ventas\n" +
129                 "3. Ver resumen total por producto\n" +
130                 "4. Volver al menú principal\n" +
131                 "Powered by EVOSYS",
132                 "Punto de Venta",
133                 JOptionPane.PLAIN_MESSAGE
134             );
135
136             if (opcion == null) break;
137             switch (opcion) {
138                 case "1": ventas.agregarVenta(productos); break;
139                 case "2": ventas.mostrarVentas(); break;
```


2. Cliente

```
Cliente.java X
Source History
1 package sistematienda;
2 public class Cliente {
3     public int idCliente;
4     public String nombre;
5     public String telefono;
6     public String direccion;
7     public String rfc;
8     public Cliente() {}
9     public Cliente(int idCliente, String nombre, String telefono) {
10         this.idCliente = idCliente;
11         this.nombre = nombre;
12         this.telefono = telefono;
13     }
14 }
```

3. Clientes

```
Cientes.java X
Source History
1 package sistematienda;
2
3 import java.sql.*;
4 import javax.swing.*;
5
6 public class Clientes {
7
8     ConexionBD objetoConexion = new ConexionBD();
9
10    public void agregarCliente() {
11        String nombre = JOptionPane.showInputDialog("Ingrese el nombre del nuevo cliente:");
12        if (nombre == null || nombre.trim().isEmpty()) return;
13
14        String telefono = JOptionPane.showInputDialog("Ingrese el teléfono:");
15
16        // Consulta SQL para insertar
17        String consulta = "INSERT INTO clientes (nombre, telefono) VALUES (?, ?)";
18
19        try {
20            PreparedStatement ps = objetoConexion.establecerConexion().prepareStatement(consulta);
21            ps.setString(1, nombre);
22            ps.setString(2, telefono);
23            ps.executeUpdate();
24            JOptionPane.showMessageDialog(null, "Cliente '" + nombre + "' guardado en MySQL.");
25        } catch (SQLException e) {
26            JOptionPane.showMessageDialog(null, "Error al guardar: " + e.toString());
27        }
28    }
29
30    public void modificarCliente() {
31        try {
32            int id = Integer.parseInt(JOptionPane.showInputDialog("Ingrese el ID del cliente a modificar:"));
33            String nuevoNombre = JOptionPane.showInputDialog("Nuevo nombre:");
34            String nuevoTel = JOptionPane.showInputDialog("Nuevo teléfono:");
35
36            String consulta = "UPDATE clientes SET nombre = ?, telefono = ? WHERE id_cliente = ?";
37
38            PreparedStatement ps = objetoConexion.establecerConexion().prepareStatement(consulta);
39            ps.setString(1, nuevoNombre);
40            ps.setString(2, nuevoTel);
41            ps.setInt(3, id);
42
43            int filas = ps.executeUpdate();
44            if (filas > 0) {
45                JOptionPane.showMessageDialog(null, "¡Cliente actualizado con éxito!");
46            } else {
47                JOptionPane.showMessageDialog(null, "No se encontró cliente con ID: " + id);
48            }
49        }
50    }
51 }
```

```

48     }
49     } catch (SQLException e) {
50         JOptionPane.showMessageDialog(null, "Error al modificar: " + e.toString());
51     } catch (NumberFormatException e) {
52         JOptionPane.showMessageDialog(null, "ID no válido.");
53     }
54 }
55
56 public void buscarCliente() {
57     String nombreBuscar = JOptionPane.showInputDialog("Nombre del cliente a buscar:");
58     if (nombreBuscar == null) return;
59
60     String consulta = "SELECT * FROM clientes WHERE nombre LIKE ?";
61
62     try {
63         PreparedStatement ps = objetoConexion.establishConexion().prepareStatement(consulta);
64         ps.setString(1, "%" + nombreBuscar + "%");
65         ResultSet rs = ps.executeQuery();
66
67         if (rs.next()) {
68             String info = "--- Datos del Cliente ---\n" +
69                 "ID: " + rs.getInt("id_cliente") +
70                 "\nNombre: " + rs.getString("nombre") +
71                 "\nTeléfono: " + rs.getString("telefono");
72             JOptionPane.showMessageDialog(null, info);
73         } else {
74             JOptionPane.showMessageDialog(null, "Cliente no encontrado.");
75         }
76     } catch (SQLException e) {
77         JOptionPane.showMessageDialog(null, "Error en búsqueda: " + e.toString());
78     }
79 }
80
81 public void mostrarClientes() {
82     String consulta = "SELECT * FROM clientes";
83     StringBuilder sb = new StringBuilder("--- LISTA DE CLIENTES EN MYSQL ---\n");
84
85     try (Statement st = objetoConexion.establishConexion().createStatement();
86          ResultSet rs = st.executeQuery(consulta)) {
87
88         boolean hayClientes = false;
89         while (rs.next()) {
90             sb.append("ID: ").append(rs.getInt("id_cliente"))
91               .append(" | Nombre: ").append(rs.getString("nombre"))

```

```

142     ps.executeBatch(); // Manda todos los nombres de una a MySQL
143     con.commit(); // Guarda los cambios permanentemente
144
145     JOptionPane.showMessageDialog(null,
146         "EVOSYS: Se han cargado " + contador + " registros correctamente.",
147         "Importación Exitosa", 1);
148
149     } catch (Exception e) {
150         JOptionPane.showMessageDialog(null,
151             "Error en el sistema EVOSYS al leer el archivo:\n" + e.getMessage(),
152             "Error de Importación", 0);
153     }
154 }
155 }
156 }

```

4.CONEXION BD

```
ConexionBD.java X
Source History
1 package sistematienda;
2 import java.sql.Connection;
3 import java.sql.DriverManager;
4 import java.sql.SQLException;
5 import javax.swing.JOptionPane;
6
7 /**
8  *
9  * @author Chavez Arellano Eduardo
10  */
11 public class ConexionBD { private Connection conectar = null;
12 // Configuración para tu base de datos "evosys"
13 private final String bd = "evosys";
14 private final String url = "jdbc:mysql://localhost:3306/" + bd;
15 private final String usuario = "root";
16 /* IMPORTANTE: Aquí abajo pon la contraseña que usas
17 para entrar a MySQL Workbench.
18 */
19 private final String password = "lunaamoR23@";
20 public Connection establecerConexion() {
21     try {
22         Class.forName("com.mysql.cj.jdbc.Driver");
23         conectar = DriverManager.getConnection(url, usuario, password);
24         System.out.println("Conexion exitosa a la base de datos: " + bd + "!");
25     } catch (ClassNotFoundException | SQLException e) {
26         JOptionPane.showMessageDialog(null, "Error al conectar: " + e.toString());
27     }
28     return conectar;
29 }
30 }
31
32
```

5.PRODUCTO

```
Producto.java X
Source History
1 package sistematienda;
2
3 public class Producto {
4     public int idProducto; // Necesario para identificar el producto en la tabla MySQL
5     public String nombre;
6     public int stock;
7     public double precio;
8
9     public Producto() {
10     }
11
12     public Producto(int idProducto, String nombre, int stock, double precio) {
13         this.idProducto = idProducto;
14         this.nombre = nombre;
15         this.stock = stock;
16         this.precio = precio;
17     }
18 }
```


6.PRODUCTOS

```
Productos.java X
Source History
1 package sistematienda;
2
3 import java.sql.*;
4 import javax.swing.*;
5
6 public class Productos {
7
8     ConexionBD objetoConexion = new ConexionBD();
9
10    public void agregarProducto() {
11        String nombre = JOptionPane.showInputDialog("Nombre del producto:");
12        if (nombre == null || nombre.trim().isEmpty()) return;
13
14        try {
15            double precio = Double.parseDouble(JOptionPane.showInputDialog("Precio:"));
16            int stock = Integer.parseInt(JOptionPane.showInputDialog("Stock inicial:"));
17
18            String consulta = "INSERT INTO productos (nombre, precio, stock) VALUES (?, ?, ?)";
19
20            PreparedStatement ps = objetoConexion.establecerConexion().prepareStatement(consulta);
21            ps.setString(1, nombre);
22            ps.setDouble(2, precio);
23            ps.setInt(3, stock);
24            ps.executeUpdate();
25
26            JOptionPane.showMessageDialog(null, "¡Producto guardado en MySQL con éxito!");
27        } catch (SQLException e) {
28            JOptionPane.showMessageDialog(null, "Error de base de datos: " + e.toString());
29        } catch (NumberFormatException e) {
30            JOptionPane.showMessageDialog(null, "Error: Ingrese números válidos para precio y stock.");
31        }
32    }
33
34    public void modificarProducto() {
35        try {
36            int id = Integer.parseInt(JOptionPane.showInputDialog("Ingrese el ID del producto a modificar:"));
37
38            String menu = "1. Nombre\n2. Stock\n3. Precio\nSeleccione opción:";
39            String opcion = JOptionPane.showInputDialog(menu);
40
41            String campo = "";
42            Object nuevoValor = null;
43
44            switch (opcion) {
45                case "1":
46                    campo = "nombre";
47                    nuevoValor = JOptionPane.showInputDialog("Nuevo nombre:");
```

```
Productos.java X
Source History

49         case "2":
50             campo = "stock";
51             nuevoValor = Integer.parseInt(JOptionPane.showInputDialog("Nuevo stock:"));
52             break;
53         case "3":
54             campo = "precio";
55             nuevoValor = Double.parseDouble(JOptionPane.showInputDialog("Nuevo precio:"));
56             break;
57         default: return;
58     }
59
60     String consulta = "UPDATE productos SET " + campo + " = ? WHERE id_producto = ?;";
61
62     PreparedStatement ps = objetoConexion.establecerConexion().prepareStatement(consulta);
63     ps.setObject(1, nuevoValor);
64     ps.setInt(2, id);
65     ps.executeUpdate();
66
67     JOptionPane.showMessageDialog(null, "Producto actualizado con éxito.");
68 } catch (SQLException e) {
69     JOptionPane.showMessageDialog(null, "Error al modificar: " + e.toString());
70 } catch (NumberFormatException e) {
71     JOptionPane.showMessageDialog(null, "Error: Ingrese un valor numérico válido.");
72 }
73 }
74
75 public void mostrarProductos() {
76     String consulta = "SELECT * FROM productos;";
77     StringBuilder sb = new StringBuilder("--- INVENTARIO EN MYSQL ---\n");
78
79     try (Statement st = objetoConexion.establecerConexion().createStatement();
80          ResultSet rs = st.executeQuery(consulta)) {
81
82         boolean hayDatos = false;
83         while (rs.next()) {
84             sb.append("ID: ").append(rs.getInt("id_producto"))
85               .append(" | ").append(rs.getString("nombre"))
86               .append(" | Stock: ").append(rs.getInt("stock"))
87               .append(" | Precio: $").append(rs.getDouble("precio")).append("\n");
88             hayDatos = true;
89         }
90
91         if (!hayDatos) {
92             JOptionPane.showMessageDialog(null, "No hay productos en la base de datos.");
93         } else {
94             JOptionPane.showMessageDialog(null, sb.toString());
95         }
96     }
```

```

101 public boolean verificarYDisminuirStock(String nombreProducto, int cantidad) {
102     String consultaBusqueda = "SELECT id_producto, stock FROM productos WHERE nombre = ?";
103     String consultaUpdate = "UPDATE productos SET stock = stock - ? WHERE id_producto = ?";
104
105     try {
106         Connection con = objetoConexion.establecerConexion();
107         PreparedStatement psBusqueda = con.prepareStatement(consultaBusqueda);
108         psBusqueda.setString(1, nombreProducto);
109         ResultSet rs = psBusqueda.executeQuery();
110
111         if (rs.next()) {
112             int stockActual = rs.getInt("stock");
113             int id = rs.getInt("id_producto");
114
115             if (stockActual >= cantidad) {
116                 PreparedStatement psUpdate = con.prepareStatement(consultaUpdate);
117                 psUpdate.setInt(1, cantidad);
118                 psUpdate.setInt(2, id);
119                 psUpdate.executeUpdate();
120                 return true;
121             }
122         }
123     } catch (SQLException e) {
124         System.err.println("Error de stock: " + e.getMessage());
125     }
126     return false;
127 }
128
129 public void cargarDesdeExcelCSV() {
130     // 1. Abrir ventana para seleccionar el archivo
131     JFileChooser selectorArchivos = new JFileChooser();
132     selectorArchivos.setDialogTitle("EVOSYS - Seleccionar Inventario (.csv)");
133     int resultado = selectorArchivos.showOpenDialog(null);
134
135     if (resultado == JFileChooser.APPROVE_OPTION) {
136         String ruta = selectorArchivos.getSelectedFile().getAbsolutePath();
137         String consulta = "INSERT INTO productos (nombre, precio, stock) VALUES (?, ?, ?)";
138
139         try (java.io.BufferedReader br = new java.io.BufferedReader(new java.io.FileReader(ruta))) {
140             Connection con = objetoConexion.establecerConexion();
141             con.setAutoCommit(false); // Optimización para EVOSYS
142             PreparedStatement ps = con.prepareStatement(consulta);
143
144             String linea;
145             int contador = 0;
146
147             while ((linea = br.readLine()) != null) {
148                 // Separamos los datos por la coma del CSV

```

```

145 while ((linea = br.readLine()) != null) {
146     // Separamos los datos por la coma del CSV
147     String[] datos = linea.split(",");
148     if (datos.length >= 3) {
149         ps.setString(1, datos[0].trim()); // Nombre
150         ps.setDouble(2, Double.parseDouble(datos[1].trim())); // Precio
151         ps.setInt(3, Integer.parseInt(datos[2].trim())); // Stock
152         ps.addBatch();
153         contador++;
154     }
155 }
156
157 ps.executeBatch();
158 con.commit();
159 JOptionPane.showMessageDialog(null, "EVOSYS: Se han importado " + contador + " productos.", "Éxito de Importación", 1);
160
161 } catch (Exception e) {
162     JOptionPane.showMessageDialog(null, "Error al leer el archivo: " + e.getMessage(), "Error EVOSYS", 0);
163 }
164
165 }
166
167 }

```


7.VENTA

```
Ventajava X
Source History
1 package sistematienda;
2 public class Venta {
3     public int idVenta; // ID propio de la venta en la base de datos
4     public int idCliente; // Para saber qué cliente compró
5     public int idProducto; // Para saber qué producto se llevó
6     public String nombreCliente;
7     public String nombreProducto;
8     public int unidades;
9     public double totalVenta;
10    public String fecha;
11    public Venta() {
12    }
13    public Venta(int idVenta, int idCliente, int idProducto, int unidades, double totalVenta, String fecha) {
14        this.idVenta = idVenta;
15        this.idCliente = idCliente;
16        this.idProducto = idProducto;
17        this.unidades = unidades;
18        this.totalVenta = totalVenta;
19        this.fecha = fecha;
20    }
21 }
```

8.VENTA CLIENTES

```
VentasClientes.java X
Source History
1 package sistematienda;
2
3 import java.sql.*;
4 import java.text.SimpleDateFormat;
5 import javax.swing.*;
6
7 public class VentasClientes {
8
9     ConexionBD objetoConexion = new ConexionBD();
10
11    public void agregarVenta(Productos productos) {
12        String clienteNombre = JOptionPane.showInputDialog("Ingrese el nombre del cliente:");
13        if (clienteNombre == null || clienteNombre.trim().isEmpty()) return;
14
15        // 1. Buscar al cliente en la base de datos
16        int idCliente = -1;
17        try {
18            String sqlCliente = "SELECT id_cliente FROM clientes WHERE nombre = ?";
19            PreparedStatement psC = objetoConexion.establecerConexion().prepareStatement(sqlCliente);
20            psC.setString(1, clienteNombre);
21            ResultSet rsC = psC.executeQuery();
22
23            if (rsC.next()) {
24                idCliente = rsC.getInt("id_cliente");
25            } else {
26                JOptionPane.showMessageDialog(null, "El cliente no existe. Por favor, regístrelo primero en el menú de Clientes.");
27                return;
28            }
29        } catch (SQLException e) {
30            JOptionPane.showMessageDialog(null, "Error al buscar cliente: " + e.getMessage());
31            return;
32        }
33
34        // 2. Buscar el producto y validar stock
35        String productoNombre = JOptionPane.showInputDialog("Ingrese el nombre del producto:");
36        int unidades = Integer.parseInt(JOptionPane.showInputDialog("Cantidad de unidades:"));
37
38        int idProducto = -1;
39        double precioUnitario = 0;
40        try {
41            String sqlProd = "SELECT id_producto, precio, stock FROM productos WHERE nombre = ?";
42            PreparedStatement psP = objetoConexion.establecerConexion().prepareStatement(sqlProd);
43            psP.setString(1, productoNombre);
44            ResultSet rsP = psP.executeQuery();
45
46            if (rsP.next()) {
47                idProducto = rsP.getInt("id_producto");
```

```

VentasClientes.java x
Source History
48         precioUnitario = rsP.getDouble("precio");
49         int stockActual = rsP.getInt("stock");
50
51         if (stockActual < unidades) {
52             JOptionPane.showMessageDialog(null, "Stock insuficiente. Disponible: " + stockActual);
53             return;
54         }
55     } else {
56         JOptionPane.showMessageDialog(null, "Producto no encontrado.");
57         return;
58     }
59 } catch (SQLException e) {
60     JOptionPane.showMessageDialog(null, "Error al buscar producto: " + e.getMessage());
61     return;
62 }
63
64 // 3. Registrar la venta y actualizar stock
65 double totalVenta = unidades * precioUnitario;
66 String fecha = new SimpleDateFormat("yyyy-MM-dd").format(new java.util.Date());
67
68 try {
69     Connection con = objetoConexion.establishConexion();
70     con.setAutoCommit(false); // Iniciar transacción
71
72     // Insertar venta
73     String sqlVenta = "INSERT INTO ventas (id_cliente, id_producto, unidades, total, fecha) VALUES (?, ?, ?, ?, ?)";
74     PreparedStatement psV = con.prepareStatement(sqlVenta);
75     psV.setInt(1, idCliente);
76     psV.setInt(2, idProducto);
77     psV.setInt(3, unidades);
78     psV.setDouble(4, totalVenta);
79     psV.setString(5, fecha);
80     psV.executeUpdate();
81
82     // Descontar stock
83     String sqlStock = "UPDATE productos SET stock = stock - ? WHERE id_producto = ?";
84     PreparedStatement psS = con.prepareStatement(sqlStock);
85     psS.setInt(1, unidades);
86     psS.setInt(2, idProducto);
87     psS.executeUpdate();
88
89     con.commit(); // Guardar cambios
90     JOptionPane.showMessageDialog(null, "Venta realizada con éxito.\nTotal: $" + totalVenta);
91 } catch (SQLException e) {
92     JOptionPane.showMessageDialog(null, "Error en la transacción: " + e.getMessage());
93 }
94 }

```

```

VentasClientes.java x
Source History
96 public void mostrarVentas() {
97     // Usamos un INNER JOIN para traer los nombres en lugar de solo los IDs
98     String consulta = "SELECT v.id_venta, c.nombre as cliente, p.nombre as producto, v.unidades, v.total, v.fecha " +
99         "FROM ventas v " +
100         "INNER JOIN clientes c ON v.id_cliente = c.id_cliente " +
101         "INNER JOIN productos p ON v.id_producto = p.id_producto";
102
103     StringBuilder sb = new StringBuilder("--- HISTORIAL DE VENTAS ---\n");
104     try (Statement st = objetoConexion.establishConexion().createStatement();
105          ResultSet rs = st.executeQuery(consulta)) {
106         while (rs.next()) {
107             sb.append("Ticket: ").append(rs.getInt("id_venta"))
108                 .append(" | ").append(rs.getString("cliente"))
109                 .append(" compró ").append(rs.getString("producto"))
110                 .append(" | Cant: ").append(rs.getInt("unidades"))
111                 .append(" | Total: $").append(rs.getDouble("total"))
112                 .append(" | Fecha: ").append(rs.getString("fecha")).append("\n");
113         }
114         JOptionPane.showMessageDialog(null, sb.toString());
115     } catch (SQLException e) {
116         JOptionPane.showMessageDialog(null, "Error: " + e.getMessage());
117     }
118 }
119
120 public void resumenVentasPorFecha() {
121     // SQL ya nos da el total sumado por producto
122     String consulta = "SELECT p.nombre, SUM(v.unidades) as total_unidades, SUM(v.total) as monto_total " +
123         "FROM ventas v " +
124         "INNER JOIN productos p ON v.id_producto = p.id_producto " +
125         "GROUP BY p.nombre";
126
127     StringBuilder sb = new StringBuilder("--- RESUMEN GENERAL DE VENTAS ---\n");
128     try (Statement st = objetoConexion.establishConexion().createStatement();
129          ResultSet rs = st.executeQuery(consulta)) {
130         while (rs.next()) {
131             sb.append("Producto: ").append(rs.getString("nombre"))
132                 .append(" | Unidades: ").append(rs.getInt("total_unidades"))
133                 .append(" | Total: $").append(rs.getDouble("monto_total")).append("\n");
134         }
135         JOptionPane.showMessageDialog(null, sb.toString());
136     } catch (SQLException e) {
137         JOptionPane.showMessageDialog(null, "Error en resumen: " + e.getMessage());
138     }
139 }
140 }

```


Interfaz gráfica del sistema EVOSYS desarrollada en Java mediante Swing



